

Image Processing & Recognition: Lecture 7 Review

Welcome to Level 7! We've moved from finding *boxes* around objects (Object Detection) to painting the exact *shape* of objects (Semantic Segmentation).

Part 1: What are you learning? (The "Human" Summary)

1. The Core Task: Semantic Segmentation

- **Goal:** Classify every single pixel in an image.
- **Result:** Instead of a box, you get a "mask" where all cat-pixels are blue, all grass-pixels are green, etc.
- **Difference from Object Detection:** Detection tells you *where* an object is (roughly). Segmentation tells you *exactly* which pixels belong to it.

2. The Problem: Shrinking Resolution

- Convolutional networks (like VGG or ResNet) use **Pooling** to shrink images and extract high-level features.
- By the end of the network, a 224×224 image might be shrunk to 7×7 .
- **Challenge:** How do we get back to a 224×224 prediction map from a tiny 7×7 feature map? We need to **Upsample**.

3. The Solution: Upsampling Techniques

- **Unpooling:** Remembering where the "max" values came from during pooling and putting them back in the same spot during upsampling.
- **Transposed Convolution:** A learnable layer that expands the image (often called "deconvolution," though that's technically incorrect). It's the reverse of a standard convolution.

4. Famous Architectures

- **FCN (Fully Convolutional Network):** Replaces the final dense layers of a classifier with convolutions, allowing it to output a heatmap instead of a single class label.
- **U-Net:** The most famous architecture for medical imaging. It has a "U" shape: it shrinks the image to capture context (contracting path) and then expands it back up (expansive path), using "skip connections" to combine high-level context with low-level details.
- **DeepLab:** Uses **Dilated Convolutions** (filters with holes in them) to capture large context without shrinking the image resolution too much.

Part 2: The Generated Solved Problem (Step-by-Step)

Since the PDF did not have a specific exercise, I have designed one based on the core mathematical operation introduced in this lecture: **Transposed Convolution** (Upsampling).

Exercise 7

We want to perform a **Transposed Convolution** operation to upsample a small input feature map.

Given:

- **Input (X):** A 2×2 matrix.

$$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

- **Kernel (K):** A 2×2 filter.

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$$

- **Stride (s):** 2 (This means we are doubling the spatial dimensions).
- **Padding (p):** 0.

Task: Calculate the Output (Y) of the transposed convolution.

Solution:

Step 1: Understand the Operation In a transposed convolution with stride 2, we take each element of the input, multiply it by the entire kernel, and place the result on the output grid, spacing them out by the stride.

- **Source: Page 15** explains: "each element in the input tensor is multiplied by the kernel... intermediate results are summed."

Step 2: Calculate Intermediate Tensors We calculate a separate 2×2 block for each input pixel.

- **Input[0,0] = 1:**

$$1 \times \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$$

Placed at top-left.

- **Input[0,1] = 2:**

$$2 \times \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix}$$

Placed at top-right (shifted by stride 2).

- **Input[1,0] = 3:**

$$3 \times \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 0 & 6 \end{bmatrix}$$

Placed at bottom-left (shifted down by stride 2).

- **Input[1,1] = 4:**

$$4 \times \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 0 \\ 0 & 8 \end{bmatrix}$$

Placed at bottom-right (shifted by stride 2).

Step 3: Assemble the Output Since stride is 2 and kernel size is 2, there is no overlap in this specific case. We just arrange the blocks.

- The output grid size will be: $(n \times s) + \max(k - s, 0)$. Here: $(2 \times 2) = 4$. So, a 4×4 output.

The grid looks like this:

$$\begin{bmatrix} 1 & 0 & 2 & 0 \\ 0 & 2 & 0 & 4 \\ 3 & 0 & 4 & 0 \\ 0 & 6 & 0 & 8 \end{bmatrix}$$

- **Source: Page 18** (Figure 2) shows visually how a stride 2 transposed convolution spaces out the outputs.

Final Result (Y):

$$Y = \begin{bmatrix} 1 & 0 & 2 & 0 \\ 0 & 2 & 0 & 4 \\ 3 & 0 & 4 & 0 \\ 0 & 6 & 0 & 8 \end{bmatrix}$$

Part 3: Your Turn (Interactive Exercises)

Practice Problem 1: Transposed Convolution with Overlap

Let's try a case where the blocks overlap, requiring addition.

Setup:

- **Input:** 1×2 vector $[1, 2]$.
- **Kernel:** 1×2 vector $[1, 1]$.
- **Stride:** 1.

Task: Calculate the output.

Solving Explanation:

1. **Block 1 (from Input 1):** $1 \times [1, 1] = [1, 1]$. Placed at index 0.
2. **Block 2 (from Input 2):** $2 \times [1, 1] = [2, 2]$. Placed at index 1 (shifted by stride 1).
3. **Summation:**
 - Index 0: 1
 - Index 1: $1 + 2 = 3$ (Overlap!)
 - Index 2: 2
4. **Result:** $[1, 3, 2]$.

Practice Problem 2: Parameter Counting (U-Net)

Setup: Consider the "bottleneck" (bottom) of a U-Net.

- **Input Feature Map:** 32×32 size, 512 channels.
- **Operation:** 3×3 Convolution (standard), doubling channels to 1024.

Task: Calculate the number of weights in this layer (ignore bias).

Solving Explanation:

1. **Kernel Size:** $3 \times 3 = 9$.
2. **Input Channels:** 512.
3. **Output Channels:** 1024.
4. **Formula:** Parameters = Kernel Size $\times$ Input Channels $\times$ Output Channels.
 $9 \times 512 \times 1024 = 4,718,592$ weights.